

Exploring React.js

A Practical Guide to Building Modern Web Interfaces

Author: Luv, Filip

Table of Contents

1. Introduction to React.js
2. Setting Up a React Project
3. Understanding JSX
4. Components in React
5. Props and State
6. Event Handling
7. Conditional Rendering
8. Lists and Keys
9. React Hooks
10. useEffect and Lifecycle Logic
11. Forms in React
12. React Router
13. API Calls in React
14. State Management Basics
15. Performance Optimization
16. React Best Practices
17. Real-World Projects
18. Conclusion

Introduction

React.js is one of the most widely used JavaScript libraries for building modern web applications. It was developed by Meta (Facebook) and is designed to create fast, interactive, and reusable user interfaces.

Instead of working with full-page reloads like traditional websites, React allows developers to build **single-page applications (SPA)** that feel smooth and responsive.

This ebook introduces React in a simple but practical way, guiding you from the basics to building real-world applications.

Chapter 1 — Introduction to React.js

React is a **component-based library** used to build user interfaces.

Why React is Popular

- Fast rendering using Virtual DOM
- Reusable components
- Strong ecosystem
- Backed by Meta
- Large community support

Where React is Used

- Social media apps
- E-commerce platforms
- Dashboards
- Mobile apps (React Native)
- SaaS products

Chapter 2 — Setting Up a React Project

To start React development, you need Node.js.

Install Node.js

[Node.js Official Website](https://nodejs.org/)

Create a React App (Vite)

```
npm create vite@latest my-app  
cd my-app  
npm install  
npm run dev
```

Project Structure

```
src/  
├── assets/  
├── components/  
├── App.jsx  
└── main.jsx
```

Chapter 3 — Understanding JSX

JSX allows writing HTML inside JavaScript.

JSX Example

```
const element = <h1>Hello React</h1>;
```

JSX Rules

- Must return a single parent element
- Use {} for JavaScript expressions
- Class → className

Chapter 4 — Components in React

Components are reusable UI building blocks.

Functional Component

```
function Welcome() {  
  return <h1>Welcome to React</h1>;  
}  
  
export default Welcome;
```

Using Components

```
import Welcome from "../Welcome";  
  
function App() {  
  return <Welcome />;  
}
```

Chapter 5 — Props and State

Props (Data Input)

```
function User(props) {  
  return <h1>{props.name}</h1>;  
}
```

State (Dynamic Data)

```
import { useState } from "react";  
  
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <button onClick={() => setCount(count + 1)}>  
      {count}  
    </button>  
  );  
}
```

Chapter 6 — Event Handling

```
function App() {  
  
  const handleClick = () => {  
    alert("Button clicked!");  
  };  
  
  return (  
    <button onClick={handleClick}>  
      Click Me  
    </button>  
  );  
}
```

Chapter 7 — Conditional Rendering

```
function App({ isLoggedIn }) {  
  return (  
    <div>  
      {isLoggedIn ? "Welcome back!" : "Please log in"}  
    </div>  
  );  
}
```

Chapter 8 — Lists and Keys

```
const items = ["React", "Vue", "Angular"];  
  
function App() {  
  return (  
    <ul>  
      {items.map((item, index) => (  
        <li key={index}>{item}</li>  
      ))}  
    </ul>  
  );  
}
```

Chapter 9 — React Hooks

Hooks allow functional components to manage state and lifecycle.

useState

Already used for state management.

useEffect

```
import { useEffect } from "react";

useEffect(() => {
  console.log("Component mounted");
}, []);
```

Chapter 10 — useEffect and Lifecycle Logic

```
useEffect(() => {
  fetch("https://api.example.com/data")
    .then(res => res.json())
    .then(data => console.log(data));
}, []);
```

Chapter 11 — Forms in React

```
import { useState } from "react";

function Form() {

  const [name, setName] = useState("");

  return (
    <input
      value={name}
      onChange={(e) => setName(e.target.value)}
    />
  );
}
```

Chapter 12 — React Router

Install Router

```
npm install react-router-dom
```

Routes Example

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Chapter 13 — API Calls in React

```
import { useEffect, useState } from "react";

function App() {

  const [data, setData] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/posts")
      .then(res => res.json())
      .then(data => setData(data));
  }, []);

  return <div>{data.length}</div>;
}
```

Chapter 14 — State Management Basics

For small apps, React state is enough.

For large apps:

- Context API
- Redux Toolkit

Context API Example

```
const UserContext = createContext();
```

Chapter 15 — Performance Optimization

React.memo

```
export default React.memo(Component);
```

Lazy Loading

```
const Dashboard = React.lazy(() => import("./Dashboard"));
```

Chapter 16 — React Best Practices

- Keep components small
- Reuse components
- Organize folders properly
- Avoid unnecessary re-renders
- Use hooks correctly

Chapter 17 — Real-World Projects

Beginner Projects

- Todo App
- Calculator
- Weather App

Intermediate Projects

- E-commerce frontend
- Blog platform
- E-library system

Advanced Projects

- Admin dashboard
- Real-time chat app
- SaaS application

Conclusion

React.js is a powerful tool for building modern web applications. By mastering its core concepts like components, state, hooks, and routing, developers can create highly interactive and scalable user interfaces.

This guide provides the foundation needed to start building real-world React applications confidently.

Final Advice

- Practice daily

- Build real projects
- Learn hooks deeply
- Understand component design
- Explore advanced state management
- Keep improving continuously

End

Exploring React.js — Modern Frontend Development Guide